

Stress Testing the AI System

Validate **SLOs** under pressure, find **bottlenecks & breakpoints** before users do, and produce a **performance report** you can defend



Test Objectives (Pick Your Targets)

Latency SLOs

p95/p99 response times (e.g., p95 < 300ms)

Throughput

Sustained QPS (e.g., 1k req/s)
your system can handle

Stability

Error rate < 0.1% during traffic spikes

Cost efficiency

\$/1k requests stays within target budget

Degradation behavior

System degrades gracefully under load/faults

Define clear objectives before testing to establish meaningful benchmarks for success.

Test Types You'll Run

1 Baseline

Single user → establish golden numbers

2 Load

Step/ramp to target QPS

3 Stress

Push beyond capacity → find the knee

4 Spike

Instant jump to $N\times$ traffic

5 Soak

Hours/days → detect leaks & performance drift

6 Chaos

Fault injection: node/pod/network/database failures

Plan Before You Blast

Workload Model

Define traffic mix and payload sizes that represent real usage

Test Data

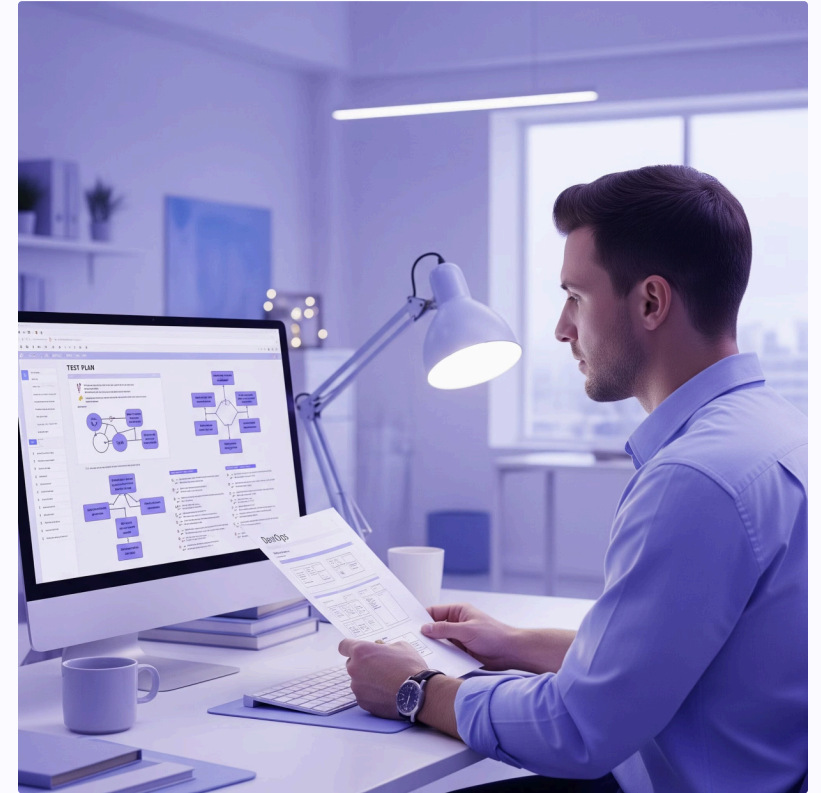
Establish cache-warm vs cache-cold scenarios for accurate comparison

Environment Configuration

Pin replicas, HPA thresholds, and node types to ensure consistency

Pass/Fail Gates

Decide on clear criteria and rollback rules before testing begins



Thorough planning prevents wasted test cycles and helps establish meaningful benchmarks.

Toolbelt (API & Infra)



Load Testing

k6, hey, vegeta, wrk

Tools for generating consistent HTTP load



Scenarios/State

Locust

Python-based tool for realistic user flows with state



Chaos Engineering

Chaos Mesh, Litmus, PowerfulSeal

Inject failures to test resilience



GPU Metrics

DCGM, tritonserver perf_analyzer

Specialized tools for GPU workload testing



Tracing

OpenTelemetry + Jaeger

Follow requests across services



Dashboards

Grafana/Prometheus, Kubecost

Visualize metrics and track costs

k6 Quickstart (HTTP API)

```
// script.js
import http from 'k6/http';
import { sleep, check } from 'k6';

export let options = {
  stages: [
    { duration: '2m', target: 200 }, // ramp
    { duration: '5m', target: 200 }, // hold
    { duration: '1m', target: 0 } // cool
  ],
  thresholds: {
    http_req_failed: ['rate<0.001'],
    http_req_duration: ['p(95)<300', 'p(99)<600']
  }
};

export default function () {
  const res = http.post(
    `${__ENV.BASE_URL}/predict`,
    JSON.stringify({text: 'hello world'}),
    { headers: { 'Content-Type': 'application/json' } }
  );
  check(res, { 'status 200': r => r.status === 200 });
  sleep(0.1);
}
```

Run: `BASE_URL=http://nlp.local k6 run script.js`



k6 provides a JavaScript API for creating sophisticated load tests with built-in metrics collection.

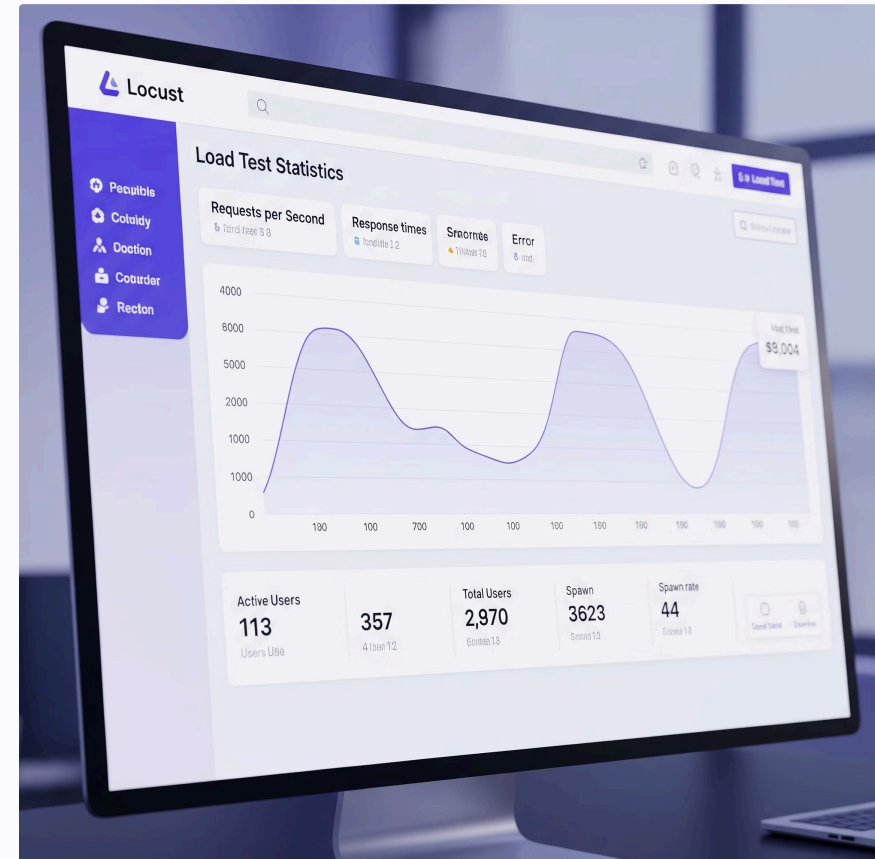
Locust (User Flows & State)

```
from locust import HttpUser, task, between
```

```
class User(HttpUser):  
    wait_time = between(0.1, 0.5)  
  
    @task  
    def predict(self):  
        self.client.post(  
            "/predict",  
            json={"text":"lorem ipsum"}  
        )
```

Run web UI: `locust -H http://nlp.local`

Locust enables Python-based test scripts that can maintain state between requests - perfect for complex user flows.



GPU & Model Throughput (Triton)

```
perf_analyzer -m my_model \  
-u triton.default.svc.cluster.local:8001 \  
-b 32 -p 1000 --input-data zero
```

Sweep **batch size** and **concurrency** to find peak throughput vs latency.



Small Batch

Lower latency but less efficient GPU utilization



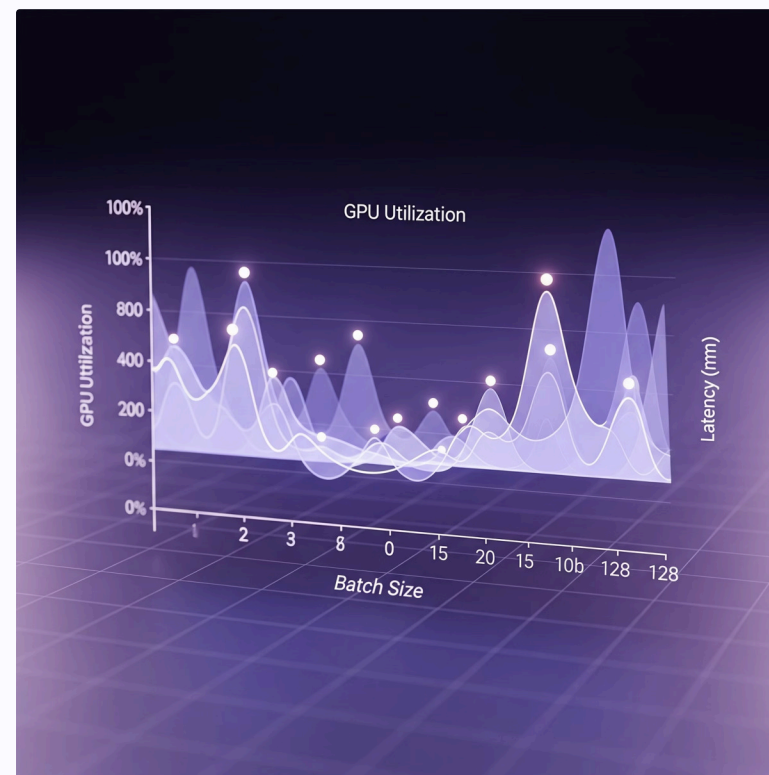
Large Batch

Higher throughput but increased p99 latency



Optimal

Balance based on your SLOs and workload

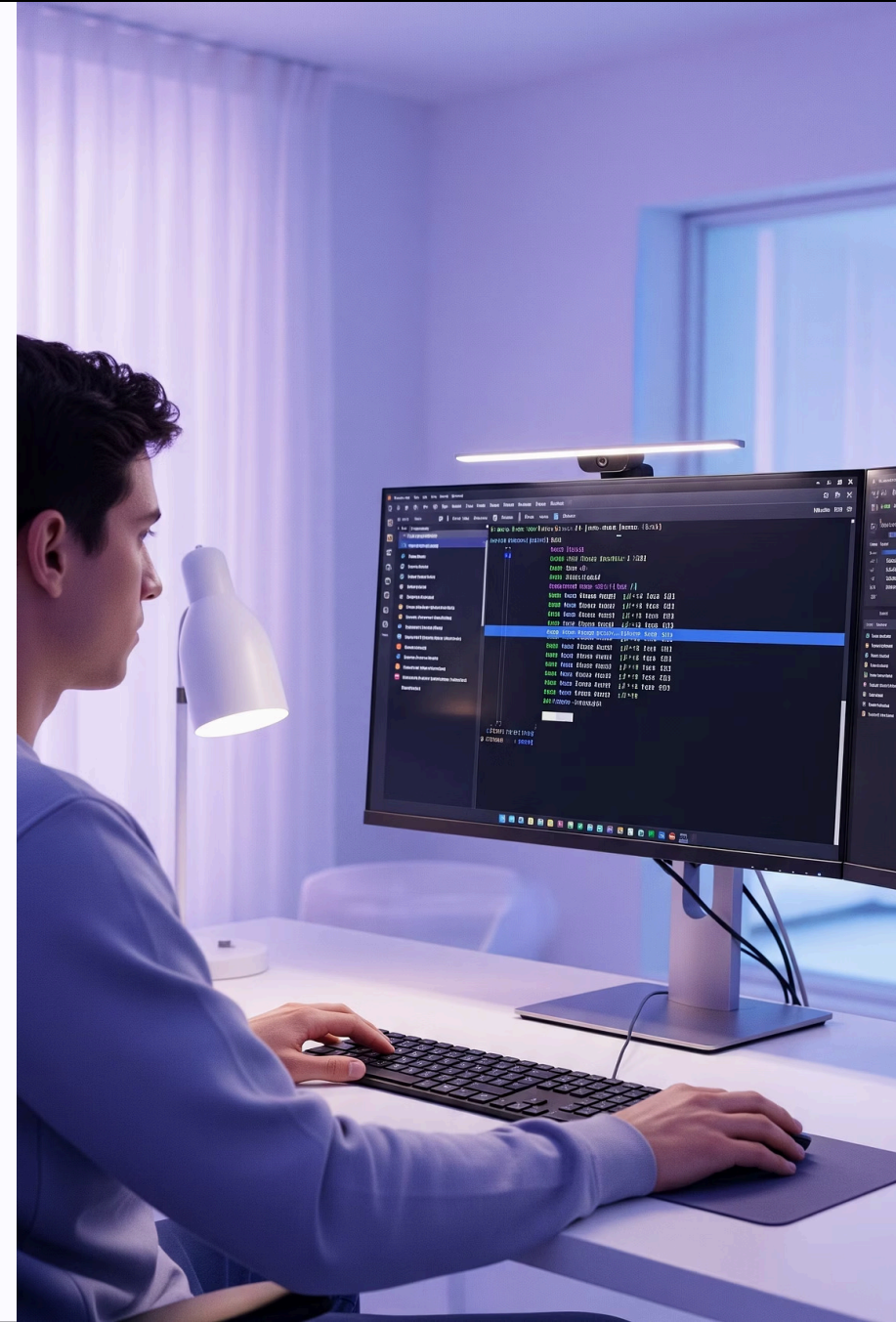


Find the sweet spot where GPU utilization is high but latency stays within acceptable bounds.

Orchestrated Test (Makefile Targets)

```
make test-baseline # k6 1 rps, golden numbers
make test-load    # k6 ramp 0→200→0
make test-stress  # k6 spike 0→1000 in 10s
make test-soak    # k6 3h at 150 rps
make chaos-podkill # chaos mesh: kill 30% pods
make report       # aggregate & export markdown/pdf
```

Use Makefiles to create a consistent, repeatable test suite that anyone on the team can run with minimal setup. This ensures consistent testing procedures and makes test results comparable across runs.



Kubernetes Readiness for Tests



Right-size replicas & set HPA targets

Configure horizontal pod autoscaling with appropriate min/max values and CPU/memory targets



Enable PodAntiAffinity and PDB

Distribute pods across nodes and set Pod Disruption Budgets to maintain availability



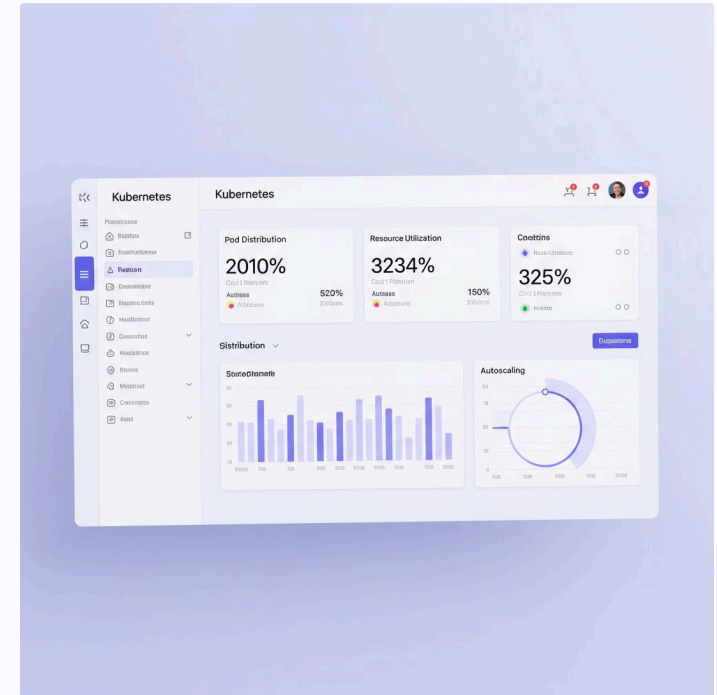
Add resource limits

Set appropriate CPU/memory limits to surface throttling early during tests



Ensure /metrics & tracing are on

Enable Prometheus metrics endpoints and distributed tracing for visibility



Properly configured Kubernetes infrastructure is essential for meaningful stress tests.

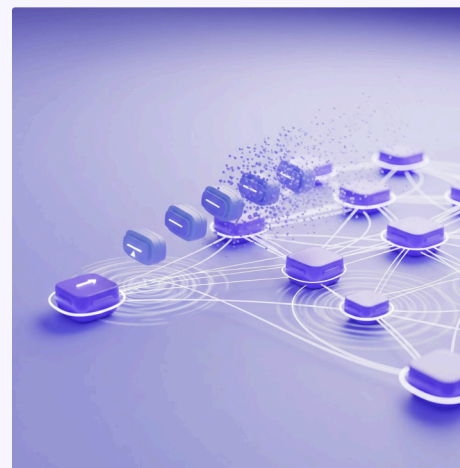
Chaos Mesh Examples

Kill 30% of inference pods for 5m

```
apiVersion: chaos-mesh.org/v1alpha1
kind: PodChaos
metadata:
  name: kill-some
  namespace: ml
spec:
  action: pod-kill
  mode: fixed-percent
  value: '30'
  selector:
    labelSelectors:
      app: "infer"
  duration: '5m'
```

Network delay 200ms p95

```
kind: NetworkChaos
spec:
  action: delay
  delay: '200ms'
  mode: all
  selector:
    labelSelectors:
      app: "infer"
```



What to Watch During Tests



API

- p50/p95/p99 latency
- Error rate, timeouts
- Success rate by endpoint



Infrastructure

- CPU utilization
- **GPU util/mem/power**
- CPU/memory throttling



Pipeline

- Kafka lag
- Airflow DAG latency
- Database connection pools



Cache

- Hit ratio
- Embedding/RAG lookups
- CDN performance



Costs

- \$/1k requests
- Idle GPU percentage
- Resource utilization efficiency

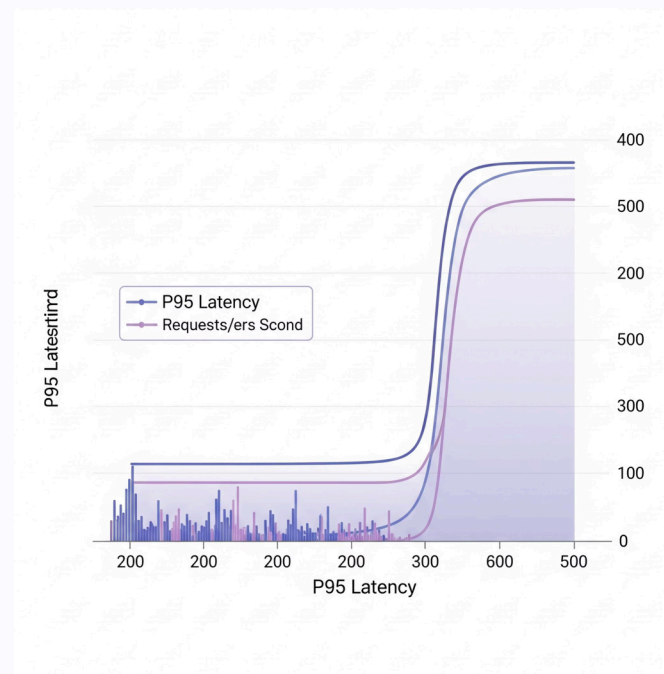
Create [dedicated dashboards](#) for test monitoring to quickly spot issues during runs.

Find the Knee (Capacity)

Increase RPS stepwise and plot **p95 latency vs RPS** to find your system's true capacity:

1. Start at low traffic and increase in steps
2. Watch for the point where p95 latency suddenly explodes
3. Look for when error rate begins to rise
4. Record this as your **capacity threshold**

Maintain **headroom** in production (e.g., run at $\leq 70\%$ of knee) to handle unexpected traffic spikes.



The "knee" marks your system's practical capacity limit - where performance degrades rapidly under additional load.

Common Bottlenecks & Fixes

High p95, low GPU util

I/O bound: Batch requests, pin threads, increase batch sizes

High 5xx rate

Memory/OOM: Reduce batch size, quantize model, add replicas

Slow first-token (LLM)

Initialization: Warmup, KV-cache, faster tokenizer, pin CPU frequency

Feature store latency

Data access: Cache hot keys, move online store near pods

Ingress limits

Connection handling: Raise worker concurrency, tune keep-alive settings



Data Scenarios Matter



- **Cache-warm vs cache-cold runs**
Test both scenarios to understand performance with/without cached data
- **Payload size distributions**
Short vs long prompts can significantly impact performance
- **Class skew for classifiers**
Imbalanced classes may affect latency and accuracy differently
- **RAG recency**
Index freshness impacts retrieval latency and quality

Real-world data distributions often reveal performance issues that synthetic tests miss.

Soak Testing (Stability)

Duration: 3–24 hours

Extended testing reveals issues that short tests miss:

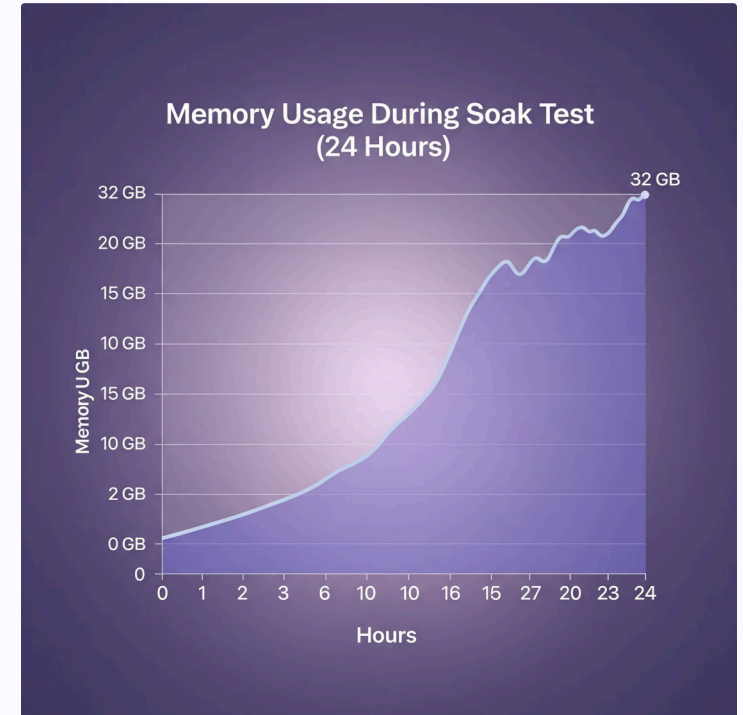
Track

- Memory growth over time
- File handles & connections
- Goroutines/threads count
- GPU temperatures

Alert on

- Latency drift
- Periodic error spikes
- **Slow resource leaks**
- Queue growth

Don't forget to rotate logs & ensure lifecycle rules (avoid disk-full conditions)



Soak tests often reveal slow resource leaks that would cause production outages weeks after deployment.

Pass/Fail Gates (Example)

Latency

p95 < **300ms** at **200 rps** for 10 min

Error Rate

Error rate < **0.1%** sustained

Cost

Cost delta < **+10%** vs last release

Stability

No **OOM/CrashLoop** events; GPU util > **40%** average

Resilience

Chaos: maintain p95 < **+25%** during 30% pod kill

Reporting Template (Copy/Paste)

Environment

Cluster size, node type, replicas, model version

Workload

Tool, script, payload details, cache state

Results

Baseline, load, stress, spike, soak (tables)

Findings

Bottlenecks with evidence (traces, panels)

Actions

Tuning items, owner, ETA

Decision

Go / No-Go with risks



A standardized report template ensures consistent communication and helps stakeholders quickly understand test results.

Example Table (Results Snapshot)

Test	RPS	p50 (ms)	p95 (ms)	p99 (ms)	5xx	Notes
Baseline	1	35	48	71	0	Golden
Load	200	120	265	390	0.03%	Meets SLO
Stress	600	240	720	1300	1.7%	Past knee
Spike	0→400	150	330	560	0.2%	Scales ok
Soak (3h)	150	135	290	510	0.05%	No leaks

Highlight [key metrics](#) that relate to your SLOs and clearly mark pass/fail conditions.

Automate in CI/CD (Nightly)

01

Run nightly **k6** job against **staging** environment

02

Export artifacts: JSON, HTML, Grafana snapshots

03

Fail build if thresholds miss; attach report to PR

04

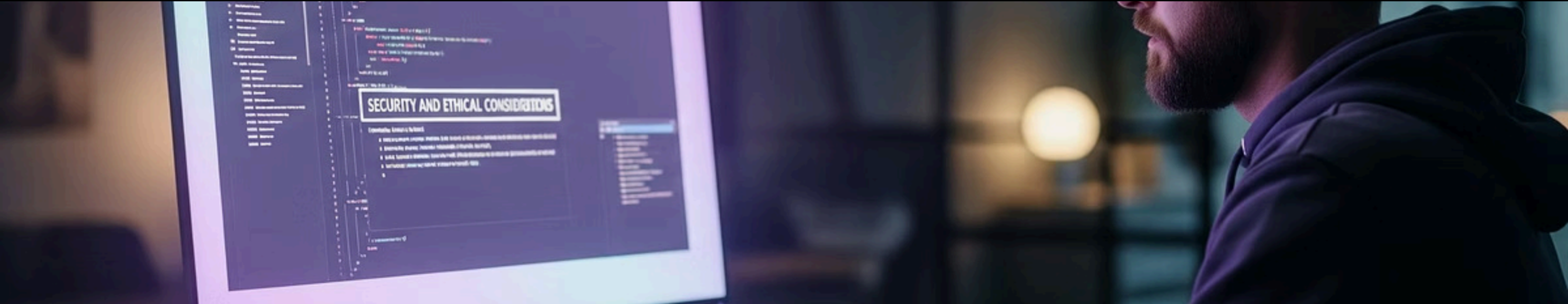
Track trends over releases (latency, \$/1k requests)

Integrate tests with your CI pipeline to catch performance regressions early:

```
# .github/workflows/nightly-perf.yml
name: Performance Test
on:
  schedule:
    - cron: '0 2 * * *' # 2 AM daily
```



Automated performance testing prevents regressions from reaching production.



Safety & Ethics



Get Approvals

Don't hit production without explicit approvals and rate limits



Tag Traffic

Mark test traffic clearly; exclude from billing if policy allows



Respect PII

Use synthetic payloads or properly anonymized data for testing

Irresponsible load testing can cause real outages and cost real money. Always follow your organization's testing protocols.

Summary

Comprehensive stress testing validates:



Validate SLOs

Confirm latency, throughput, and stability targets



Verify Cost

Ensure performance meets budget constraints



Test Resilience

Verify graceful degradation during failures

Use **k6/Locust + Chaos Mesh + Triton perf_analyzer** to create a complete test suite

Follow the cycle: Measure, identify the knee, fix bottlenecks, **re-test**

Produce a clear **Go/No-Go** report for stakeholders





Discussion Prompt

"Which limit do you expect to hit first in your Capstone: model compute, feature store latency, or network/ingress? What will you test first to prove it?"

Think about your specific application architecture and where your current bottlenecks might be.